



CIN0136

Desenvolvimento de Software

Desenvolvimento de Software

Prof. Vinicius Cardoso Garcia

vcg@cin.ufpe.br :: @vinicius3w :: assertlab.com

<http://viniciusgarcia.me>



**Centro de
Informática**
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO

Licença do material

Este Trabalho foi licenciado com uma Licença



**Atribuição-NãoComercial-
Compartilha Igual 4.0 Internacional
(CC BY-NC-SA 4.0)**

Mais informações visite

<https://creativecommons.org/licenses/by-nc-sa/4.0/deed.pt> BR



Engenharia de Software em Dimensões

É um material abrangente, didático e atualizado que apresenta a Engenharia de Software como uma disciplina multidimensional. Cada capítulo foca em uma dimensão prática, conectando teoria e realidade profissional com clareza, profundidade e propósito.

Prof. Vinicius Cardoso Garcia

<https://bit.ly/esdbook>

Engenharia de Software em Dimensões

Da base conceitual à prática profissional



Vinicius Cardoso Garcia



Ponto de Commit

O podcast figital da disciplina (IF977) Engenharia de Software do curso de bacharelado em Sistemas de Informação do Centro de Informática (CIn) da Universidade Federal de Pernambuco (UFPE).

<https://go.uaite.xyz/ponto-de-commit>

Ponto de Commit



O podcast figital da disciplina de Engenharia de Software



Engenharia de Software Moderna

Engenharia de Software Moderna é um livro destinado a alunos de cursos de graduação em Computação. Ele pode ser lido também por profissionais que buscam um conhecimento horizontal da área.

Prof. Marco Tulio Valente

<https://engsoftmoderna.info>

@engsoftmoderna

ENGENHARIA — DE — SOFTWARE MODERNA

Princípios e práticas para
desenvolvimento de software
com produtividade



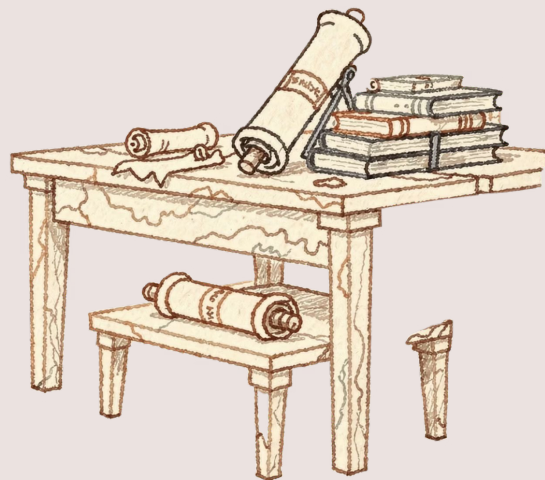
MARCO TULIO VALENTE





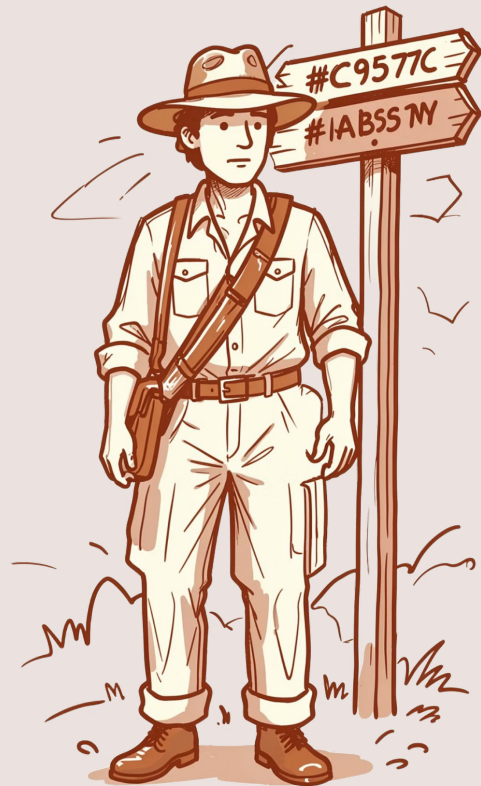
Documentação e Entrega de MVP

O legado do projeto começa onde o código termina, garantindo que o conhecimento sobreviva à equipe que o criou





**"Se a equipe
fosse embora
amanhã, outra
conseguiria
continuar?"**



O Custo da Falta de Documentação

Retrabalho

Decisões já tomadas são revividas do zero, consumindo tempo e orçamento.

Dívida Técnica

Código sem contexto se torna um labirinto impossível de manter.

Paralisa Operacional

Novos integrantes ficam bloqueados sem um ponto de partida confiável.



Por que todo mundo sabe que documentação é importante, e ninguém faz

Verdade 1

É sempre a **última coisa** a entrar no sprint, sempre tem algo mais urgente.

Verdade 2

É a **primeira a ser cortada** quando o prazo aperta. Sem cerimônia.

Verdade 3

O README tende a ter o conteúdo mais famoso da engenharia:

```
# TODO: add docs
```

"Documentação é o conjunto de artefatos que tornam o conhecimento sobre o sistema acessível além de quem o criou."
— Engenharia de Software em Dimensões, Cap. 19

A pergunta que muda a lógica: em vez de *"o que documento?"*, pergunte: **"o que outra pessoa precisaria saber para trabalhar com isso?"**



O risco que a PetFood identificou cedo

A PetFood implementou Code Review não só pela qualidade técnica. Um dos benefícios explicitamente valorizados por Bruno e Ana foi a **socialização do conhecimento**:

"Ao revisar o código de autenticação, Bruno aprende como a lógica de segurança da PetFood está sendo construída. Isso socializa o conhecimento, reduz a dependência de especialistas e o '**fator ônibus**'. o risco de perder conhecimento crítico se uma pessoa sair da equipe."
— Engenharia de Software em Dimensões, Cap. 2, seção 2.5.1

⚠ **Pergunta para a turma:** No projeto de vocês, qual membro da equipe, se saísse agora, deixaria um buraco de conhecimento irrecuperável? O que está só *na cabeça* dessa pessoa?



O Ecossistema de Documentação



Documentação como ativo vivo

Documentação não é arquivo morto, ela é versionada no Git junto com o código, evoluindo com o projeto a cada commit e pull request.

O tripé da sustentabilidade

- **README:** o *quê* do projeto
- **ADR:** o *porquê* das decisões
- **Spec:** o *contrato* de comportamento



README: A Porta de Entrada

O primeiro arquivo que qualquer pessoa lê, deve responder às perguntas essenciais em minutos.

1

Visão Geral

Qual problema é resolvido, qual a solução proposta e quem é o público-alvo.

2

Guia de Instalação

Pré-requisitos, variáveis de ambiente, comandos passo a passo e como rodar localmente.

3

Como Contribuir

Normas de estilo de código, fluxo de branches e links para templates de PR e Issues.



O que está errado aqui?

Exemplo A — para análise da turma

```
# Projeto
Sistema desenvolvido para o stakeholder X.

## Tecnologias
- React
- Node.js
- PostgreSQL

## Como rodar
npm install
npm start
```

O que está faltando?

- Qual versão do Node.js?
- Precisa de banco configurado? Como?
- Tem variáveis de ambiente? Quais?
- Como rodo os testes?
- Para que serve esse sistema, afinal?

⊗ Este README não transfere nenhum conhecimento real. É decoração.



O que um README completo parece

```
# Sistema de Gestão de Reservas – ONG Recife Verde  
Plataforma web para gerenciamento de reservas de espaços  
comunitários · CIN0136/UFPE 2026.1
```

```
## Pré-requisitos  
Node.js 20+ | PostgreSQL 15+  
Copie .env.example → .env e preencha as variáveis
```

```
## Como rodar localmente  
npm install  
npm run db:migrate && npm run db:seed  
npm run dev → http://localhost:3000
```

```
## Como testar  
npm run test          # unitários (Vitest)  
npm run test:e2e      # aceitação (Playwright)  
npm run test:coverage # relatório de cobertura
```

```
## Estrutura  
src/frontend → React + Vite  
src/backend  → Express + Prisma  
docs/adrs    → Decisões arquiteturais  
docs/c4      → Diagramas de arquitetura
```

👉 **O que mudou:** contexto real, versões específicas, variáveis de ambiente, comandos de teste separados, estrutura de pastas explicada. Qualquer pessoa consegue rodar do zero.



Arquitetura e Decisões (ADRs)



Por que ADRs importam?

Architecture Decision Records criam um histórico imutável de **quem decidiu, quando e por quê**, evitando que novas equipes revertam padrões vitais por desconhecimento.

- Um bom ADR responde: por que escolhemos X em vez de Y, e quais trade-offs aceitamos.



Uma decisão não documentada é um débito oculto

Architecture Decision Record (ADR) é um registro curto de uma decisão de design importante. Use sempre que alguém puder perguntar "por que está assim?" no futuro.

Formato mínimo

```
## ADR 001 - [Título]
**Data:** DD/MM/AAAA
**Status:** Aceita

**Contexto:**
Por que essa decisão precisou ser tomada?

**Decisão:**
O que foi decidido?

**Consequências:**
O que muda? O que fica mais fácil?
O que fica mais difícil?
```

Exemplos do seu projeto

- Por que usaram esse banco de dados e não outro?
- Por que organizaram as rotas dessa forma específica?
- Por que abandonaram a biblioteca X e adotaram a Y?



Comentários que explicam o porquê

O código já diz *o que* ele faz. Só você sabe *por que* ele faz assim. Esse é o conhecimento que precisa ser registrado.

⊗ ✗ Comentário inútil

```
// incrementa o contador  
count++;
```

Isso é ruído. O código já é autoexplicativo. Não acrescenta nada.

☑️ ✓ Comentário que transfere conhecimento

```
// A API do parceiro retorna HTTP 200  
// mesmo em erros de negócio – verificamos  
// o campo 'success' explicitamente  
// para não confiar só no status code.  
if (!response.success) { ... }
```

Isso é contexto que nenhuma outra pessoa teria. Essencial.



O checklist de entrega, o que verificar antes do release

README

- Descrição do projeto (problema + solução + organização parceira)
- Pré-requisitos com versões específicas
- Comandos testados por alguém que nunca viu o projeto
- Como rodar os testes
- Estrutura de pastas explicada

ADRs

- Pelo menos 3 decisões documentadas
- Escolha do banco justificada
- Organização não-óbvia do código registrada

Diagramas C4

- Nível 1 (Contexto) atualizado
- Nível 2 (Container) atualizado
- Diagramas no repositório, não só na apresentação

Código

- Sem `// TODO` aberto na entrega
- Sem código comentado sem justificativa



Aplique o checklist no seu repositório

Esta não é uma atividade para fazer tudo de vez, é para **mapear o que falta**. Construam uma lista de tarefas concreta para os próximos 2 dias.



~20 minutos

Em equipe, abram o repositório e passem item por item do checklist.



Output esperado

Uma lista priorizada do que precisa ser feito antes de quinta-feira, com responsável para cada item.



Pergunta guia: "Se vocês fossem embora do projeto amanhã, o que seria mais difícil de entender?" Esse é o lugar que **mais** precisa de documentação.





A Nova Era: Documentação para Agentes de IA

Ferramentas como **Claude Code** e **Cursor** leem o contexto do projeto antes de gerar qualquer código, tornando a documentação um input direto de qualidade.



CLAUDE.md e AGENTS.md

O onboarding permanente da sua IA

Esses arquivos na raiz do repositório descrevem o stack técnico, convenções de código, regras de negócio e restrições, funcionando como um briefing contínuo para agentes de IA.

- ✓ Resultado: menos alucinações, mais alinhamento com as escolhas do time e respostas mais contextualizadas.

O que incluir

- Linguagens, frameworks e versões utilizadas
- Padrões de nomenclatura e estrutura de pastas
- Regras de negócio críticas e invariantes
- O que **não** deve ser alterado



O Que Documentar: Checklist de Entrega



README completo

Visão geral, instalação, uso e guia de contribuição.



Diagramas de Arquitetura

Fluxo de dados, infraestrutura e integrações externas.



Modelos de Dados

Especificação de entidades, relacionamentos e casos de uso.



Templates de Issues

Formulários padronizados para relato de bugs e sugestão de features.



Templates de Issues e Pull Requests



Bug Report

- Descrição do comportamento esperado vs. atual
- Passos para reproduzir o problema
- Ambiente (OS, versão, browser)
- Logs e screenshots relevantes



Feature Request / Pull Request

- Problema que a feature resolve
- Solução proposta e alternativas consideradas
- Critérios de aceite (Definition of Done)
- Checklist de revisão de código





A Transição: Handover

Entregar um MVP é mais do que transferir repositório. É a transmissão formal do **conhecimento operacional**, o que só existe na cabeça das pessoas.



Os Três Pilares do Handover



Treinamento de Operadores

Capacitação prática da equipe que vai manter e evoluir o sistema.



Hypercare

Período de suporte intensivo pós-entrega, com SLA reduzido e time original disponível.

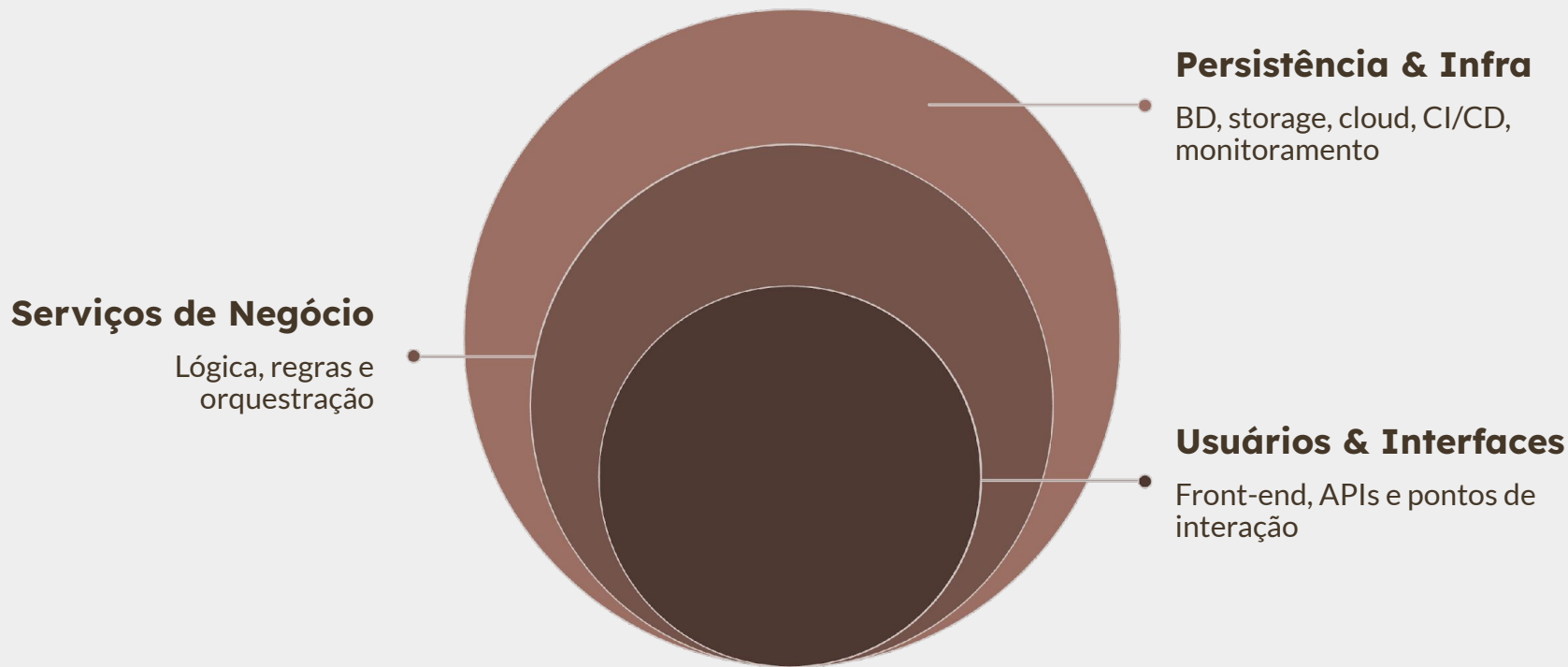


Sign-Off Formal

O aceite formal do cliente ou gestor encerra a responsabilidade da equipe de desenvolvimento.



Diagrama de Arquitetura: O Que Incluir



Um diagrama claro elimina horas de investigação para qualquer pessoa que chegue ao projeto depois.

O fator tempo na documentação

O projeto de vocês se encaixa no nível **projeto acadêmico com impacto real**, uma ONG vai depender deste software por anos. Isso muda a responsabilidade da equipe.



O produto não acaba quando o código funciona

1

Código funcionando

As funcionalidades implementadas passam nos testes automatizados.

2

Testes passando

Cobertura adequada (unitários e de aceitação) comprovando que o sistema faz o que promete.

3

Documentação transfere conhecimento

Qualquer pessoa consegue entender, rodar e evoluir o sistema sem precisar da equipe original.

4

Produto entregue

Pode ser entregue, mantido e evoluído, agora e daqui a 2 anos.



Lições Aprendidas



Retro rápida ao final do projeto

Reserve 30 minutos para capturar o conhecimento tácito antes que ele se perca.

-  O que funcionou bem
-  O que falhou e por quê
-  Recomendações para o próximo ciclo

☐ Mantenha um **One-Page Summary** de lições, fácil de compartilhar e impossível de ignorar.



Onde Arquivar a Documentação



Notion / Confluence

Wikis colaborativas para documentação viva, acessível por toda a organização.



Repositório Git

README, ADRs e AGENTS.md versionados junto com o código.



Google Drive / SharePoint

Documentos formais, diagramas exportados e registros de sign-off.



Checklist Final de Encerramento

Documentação

- ✓ README atualizado e revisado
- ✓ ADRs registrados e versionados
- ✓ Diagramas de arquitetura exportados
- ✓ CLAUDE.md / AGENTS.md criados
- ✓ Templates de Issues e PRs configurados

Transição e Encerramento

- ✓ Treinamento dos operadores concluído
- ✓ Período de Hypercare definido e comunicado
- ✓ Sign-Off formal coletado
- ✓ SLA e contatos de suporte documentados
- ✓ Projeto marcado como **CLOSED** no PMO



A Pergunta Respondida

Sem documentação

Nova equipe enfrenta semanas de arqueologia no código, decisões refeitas e erros repetidos.

Com documentação

Onboarding em dias, não semanas. Contexto preservado. Projeto escalável e sustentável.





O Código é Efêmero. O Conhecimento é Eterno

Documentar é um ato de respeito com o futuro da sua equipe, e com qualquer pessoa que um dia tocar neste projeto.



Prepare para a Escalabilidade, não para a sua Permanência

O que você deixa para trás

- Um sistema que outros entendem e evoluem
- Decisões fundamentadas e rastreáveis
- Uma cultura de documentação como norma

O que você leva com você

- Reputação de profissional que pensa no coletivo
- Projetos que sobrevivem à sua saída
- Credibilidade técnica e organizacional



Resumo: Os 5 Entregáveis Inegociáveis



Checklist de encerramento



Lições aprendidas (One-Pager)



Diagramas de arquitetura



ADRs versionados

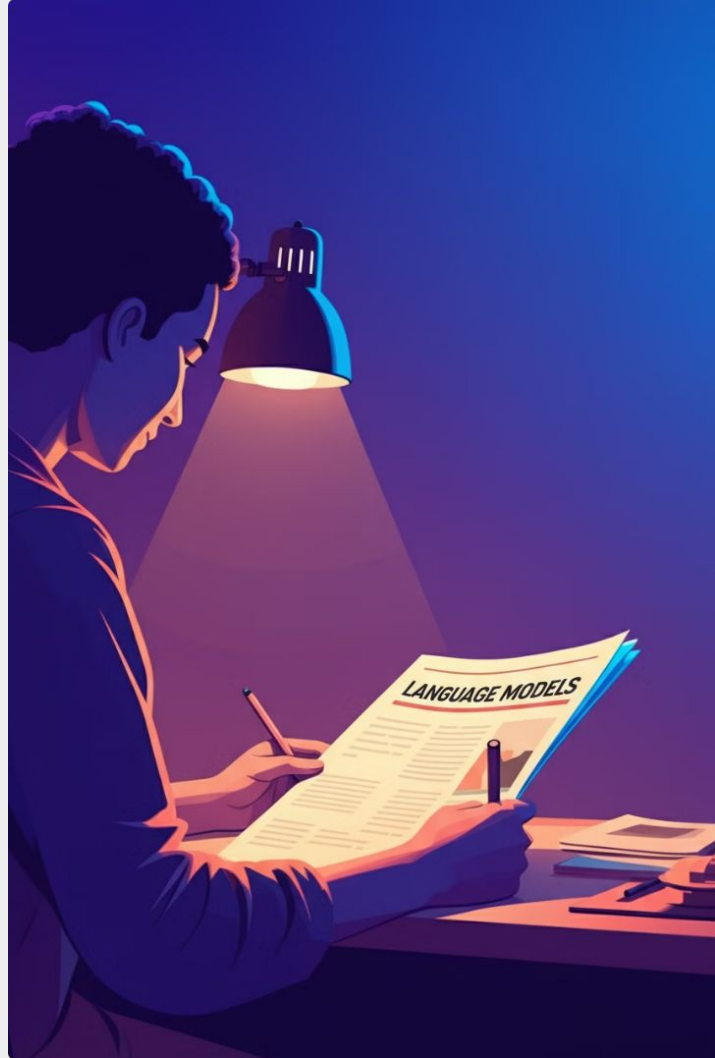


README completo e atualizado



Próximos passos

- Eng. Software em Dimensões
 - Cap. 2, seção 2.5.1 — Fator ônibus e socialização do conhecimento via Code Review.
 - Cap. 3 (seções 3.1–3.7): Versionamento, Git, PRs, Code Review, licenças
 - Cap. 14 (seções 14.1–14.8): Arquitetura, C4 Model, qualidade arquitetural
 - Cap. 19 (seções 19.1–19.4): Manutenção, refactoring, dívida técnica, documentação
- Eng. Software Moderna
 - Cap. 9 — Refactoring, Catálogo, code smells, prática
 - Cap. 10 — DevOps, Controle de versões, integração contínua



contacts

vcg@cin.ufpe.br
[@vinicius3w](https://www.instagram.com/vinicius3w)
viniciusgarcia.me

academy

go.uaite.xyz/vcg-es
go.uaite.xyz/vcg-microservices
go.uaite.xyz/vcg-devops
go.uaite.xyz/vcg-esaia
go.uaite.xyz/vcg-deso

Desenvolvimento de Software (CIN0136)

Documentação e Entrega de MVP